

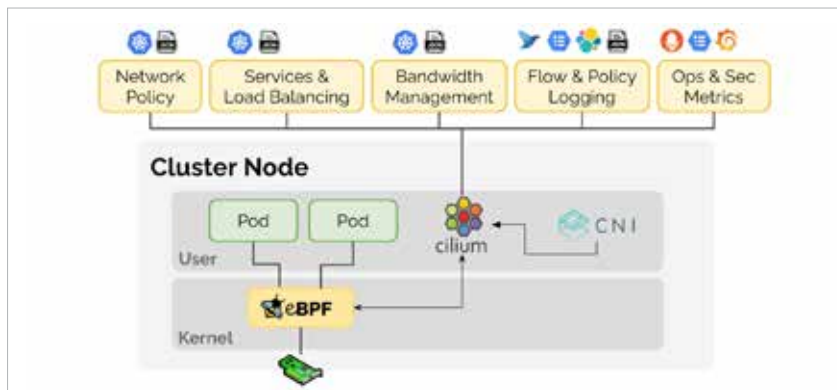


Figure 3: eBPF integration with the Kubernetes

Observability with eBPF in Kubernetes

Observability in Kubernetes environments is crucial for monitoring performance, troubleshooting issues, and ensuring efficient resource utilisation. Traditional observability tools rely on kernel-level metrics and logs, which often provide limited visibility into the system's real-time state. eBPF dramatically improves observability by capturing detailed telemetry data directly from the kernel, giving operators deep insights into the behaviour of applications and system components.

Real-time monitoring of system events

eBPF enables real-time monitoring of system calls, network packets, and process activities. By attaching eBPF programs to various kernel hooks, you can capture detailed metrics about CPU usage, disk I/O, memory usage, and network traffic in a non-intrusive way. These metrics help identify performance bottlenecks, diagnose latency issues, and troubleshoot errors in Kubernetes clusters.

In addition to traditional monitoring tools, eBPF can be integrated with modern observability platforms like Prometheus and Grafana to provide visualisations of detailed, low-level metrics captured from the kernel. This granular level of observability is essential for

performance tuning in complex, multi-node Kubernetes environments.

Application tracing and debugging

eBPF allows developers to trace and debug applications in Kubernetes clusters with unprecedented precision. With eBPF-based tracing, you can follow the entire lifecycle of a request as it travels through various services and containers within the Kubernetes cluster. This allows for detailed analysis of where latency is introduced or where resource bottlenecks occur.

BPFTrace, a high-level tracing tool built on eBPF, provides a simple way to write scripts that capture system events and metrics, such as system calls, network activity, and process events. By running BPFTrace in Kubernetes clusters, developers can debug issues related to resource contention, network slowdowns, or performance degradation at the kernel level, all without modifying the underlying application code.

Security auditing and intrusion detection

With eBPF, you can also perform real-time security auditing by capturing data about suspicious activities in the system, such as unauthorised access attempts or unusual system behaviour. This provides a robust method for detecting potential security

breaches, even within highly dynamic Kubernetes environments.

By integrating eBPF with security tools like Falco, an open source project for real-time threat detection, Kubernetes administrators can continuously monitor system events for anomalies and malicious behaviour. eBPF provides deep visibility into network traffic and system operations, allowing for quicker detection and response to security incidents.

Practical implementation and use cases

The integration of eBPF into Kubernetes can be implemented using various tools and frameworks, such as Cilium for networking and BPFTrace for observability. Given below are some practical steps and real-world use cases showcasing the power of eBPF in Kubernetes.

Setting up eBPF with Cilium for networking: To enhance Kubernetes networking with eBPF, you can install and configure Cilium as a CNI plugin in your cluster. Cilium leverages eBPF to provide high-performance networking, advanced security, and network policy enforcement.

Here's a simplified guide on how to set up Cilium.

Install Cilium in your Kubernetes cluster:

```
helm repo add cilium https://helm.cilium.io/
helm install cilium cilium/cilium
--version 1.11.0 --namespace kube-system
```

Enable eBPF-based load balancing and network policies:

```
helm upgrade cilium cilium/cilium
--namespace kube-system --set global.
nodePort.enabled=true --set global.ebpf.
enabled=true
```

Apply network policies to secure communication between pods:

```
kubectl apply -f my-network-policy.yaml
```